G. S. Mandal's

Maharashtra Institute of Technology, Chhatrapati

Sambhajinagar (An Autonomous Institute)

Department Electronics & Computer Engineering

Dr. BABASAHEB AMBEDKAR MARATHWADA UNIVERSITY,

CHHATRAPATI SAMBHAJINAGAR (M.S.)

A Mini Project Report on

Smart Room Climate Monitoring System Using STM32

Submitted by

Sahil Dubole (A-64022)

Under the guidance of

Mr. Shailesh Biradar

In partial fulfilment of the award of Bachelor of Technology in Electronics & Computer Engineering



Department of Electronics & Computer Engineering

Maharashtra Institute of Technology, Chhatrapati Sambhajinagar (Aurangabad)

[2026-27]

CERTIFICATE

This is to certify that mini project entitled **Smart Room Climate Monitoring Using STM32**, submitted by **Sahil Dubole** is the Bonafide work completed under my supervision and guidance in partial fulfilment for the award of Bachelor of Technology in (**Electronics and Computer Engineering**), Maharashtra Institute of Technology under Dr. Babasaheb Ambedkar Marathwada University, Chhatrapati Sambhajinagar (Aurangabad) (M.S.).

Place: Chhatrapati Sambhajinagar (Aurangabad)

Date:

Mr. Shailesh Biradar

Course Coordinator

Submitted by

Sahil Shrikrushna Dubole

Maharashtra Institute of Technology
B.Tech. in Electronics / Electronics and Computer Engineering
Academic Year 2026–2027

Project: Embedded Systems / Microcontroller Applications

TABLE OF CONTENTS

CHAPTERS / TITLE	PAGE NO.
ABSTRACT	i
1. INTRODUCTION	1
1.1 Need of the Project	1
1.2 Objectives	2
2. SYSTEM DESIGN	3
2.1 Overview of the System	3
2.2 Main Components of the System	4
2.3 Functional Block Diagram	5
2.4 Working of the System	6
2.5 Software Module Design	7
2.6 Design Considerations	8
3. CONNECTIONS	9
3.1 DHT11 Sensor Connections	9
3.2 LCD Connections	10
3.3 Communication Between Components	11
3.4 Power Supply and Ground Connections	12
3.5 Precautions	13
4. SOFTWARE IMPLEMENTATION	14
4.1 Libraries Required	14
4.2 Pin Configuration	15
4.3 Program Code	16
5. RESULTS AND OBSERVATIONS	18
5.1 Observed Readings	18
5.2 Output Analysis	19
5.3 Result	19
6. SYSTEM FEATURES AND LIMITATIONS	20
6.1 Advantages	20
6.2 Applications	21
6.3 Limitations	21
7. CONCLUSION AND FUTURE SCOPE	22
7.1 Conclusion	22
7.2 Future Scope	22
REFERENCES	23

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
Fig. 2.1	Block Diagram of Smart Room Climate Monitoring System	5
Fig. 2.2	System Architecture of the Proposed System	6
Fig. 2.3	Flowchart of System Operation	7
Fig. 3.1	STM32 NUCLEO-L031K6 Development Board	8
Fig. 3.2	DHT11 Temperature and Humidity Sensor	9
Fig. 3.3	16×2 LCD Display with I2C Module	10
Fig. 3.4	Circuit Diagram of the Complete System	11
Fig. 4.1	Arduino IDE Software Setup	13
Fig. 4.2	Program Uploading to the STM32 Board	14
Fig. 5.1	LCD Display Showing Temperature and Humidity Readings	16
Fig. 5.2	LCD Display Showing WARM Status	16

ABSTRACT

Monitoring room temperature and relative humidity is useful for maintaining comfortable indoor conditions and identifying warm or humid environments. This mini project presents a compact room climate monitoring system built around an STM32 NUCLEO-L031K6 development board, a DHT11 temperature-and-humidity sensor, and a 16×2 character LCD fitted with an I2C interface backpack.

The DHT11 sensor is connected to pin PB0 of the STM32 board. The LCD communicates with the microcontroller through I2C, using PB6 as SCL and PB7 as SDA in the configured setup. The program periodically requests a sensor reading, checks the received data, displays temperature and relative humidity, and classifies the room condition using a simple threshold rule. In the tested configuration, a temperature reading of 31°C and relative humidity of 52% produced the status “WARM” because the temperature threshold was set to 30°C.

The project demonstrates embedded-system fundamentals including GPIO, timing-sensitive sensor communication, I2C communication, LCD interfacing, conditional logic, and microcontroller programming using the Arduino IDE with the STM32 core. It is a low-cost educational prototype and can be extended with data logging, wireless monitoring, alarms, or additional environmental sensors.

1. INTRODUCTION

Room temperature and humidity influence comfort, storage conditions, electronic equipment operation, and indoor environmental quality. A simple monitoring device can display these parameters continuously and provide a quick indication when the room becomes warm or humid.

This project uses an STM32 microcontroller as the processing unit. The DHT11 sensor measures temperature and relative humidity, while a 16×2 I2C LCD presents the values in a human-readable form. A small embedded program coordinates sensor sampling, validation, threshold checking, and display updates.

1.1 Need of the Project

Manual measurement with a handheld thermometer or hygrometer is not continuous and may require repeated checking. A small electronic monitor provides repeated readings on a display without requiring a computer to remain connected. It also gives students practical experience with sensor interfacing, microcontroller pin configuration, serial protocols, and embedded C/C++ programming.

The prototype is intended for learning and demonstration. The DHT11 is a low-cost sensor with limited accuracy and response speed, so it should not be treated as a calibrated instrument for critical industrial or medical decisions.

1.2 Objectives

• Measure room temperature and relative humidity using a DHT11 sensor.

• Read the sensor through an STM32 NUCLEO-L031K6 development board.

• Display the measured values on a 16×2 LCD using an I2C backpack.

• Apply simple threshold logic to show a room-condition message such as NORMAL, WARM, or HUMID.

• Build and test the firmware using the Arduino IDE and STM32 board support package.

• Document the circuit connections, software logic, observations, limitations, and possible improvements.

2. SYSTEM DESIGN

2.1 Overview of the System

The system consists of three main hardware blocks: the DHT11 sensor, the STM32 microcontroller board, and the I2C LCD. The sensor converts ambient temperature and humidity into a digital data frame. The STM32 reads and interprets the frame, applies threshold conditions, and sends text to the LCD. The board is programmed from a computer through its supported ST-LINK/OpenOCD upload path.

The current implementation uses DHT11 data on PB0, I2C SCL on PB6, and I2C SDA on PB7. The exact pin mapping must match the board variant and the firmware configuration selected in the Arduino IDE.

2.2 Main Components of the System

Component	Purpose / Description
STM32 NUCLEO-L031K6	Low-power microcontroller board; reads sensor data and controls the LCD.
DHT11 sensor	Provides digital temperature and relative-humidity readings.
16×2 character LCD with I2C backpack	Displays temperature, humidity, and the status message.
Jumper wires / breadboard	Provide signal, supply, and ground connections.
USB cable / suitable supply	Powers the development board and supports programming/debugging.
Arduino IDE with STM32 core	Used to write, compile, and upload the firmware.

2.3 Functional Block Diagram

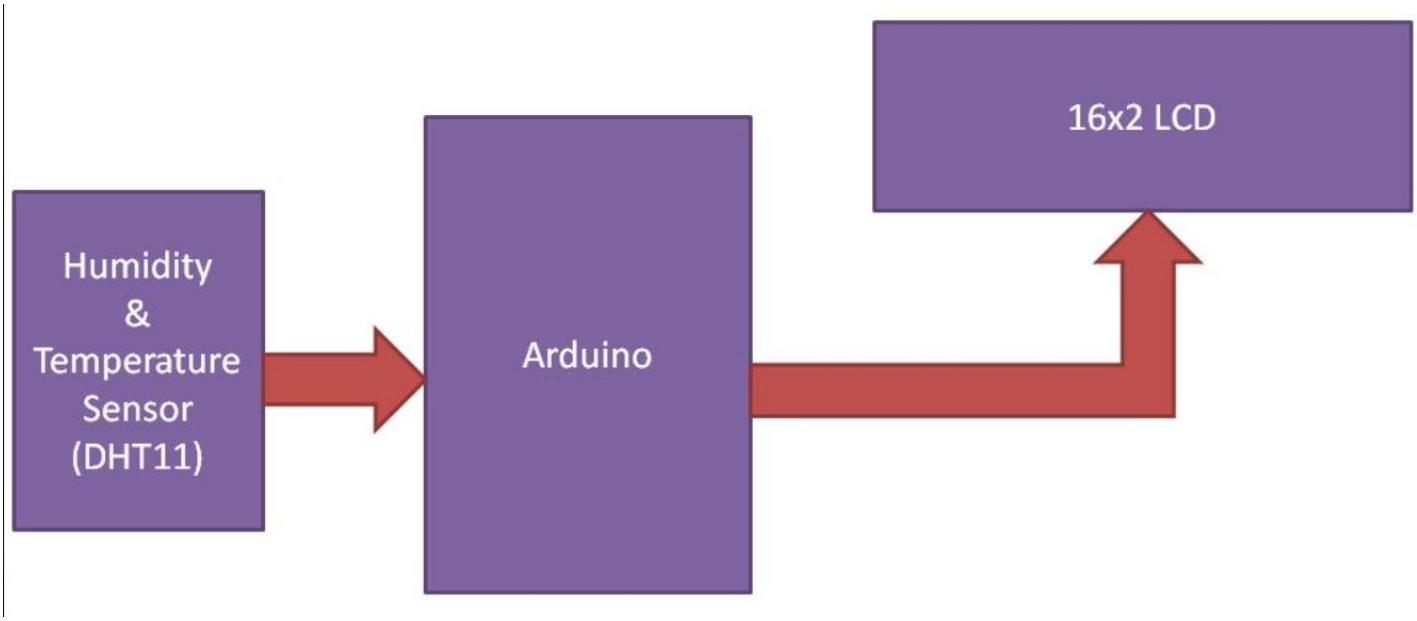


Figure 2.1: Functional block diagram of the room climate monitoring system.

2.4 Working of the System

Step 1: Power Supply and Initialization

The STM32 NUCLEO-L031K6 board receives power through its USB connection or another supported power source.

When the board is powered on, the program starts executing. It initializes the DHT11 sensor interface and the I2C

communication used by the LCD.

The LCD is initialized so that it can display characters on its two rows.

Step 2: Temperature and Humidity Measurement

The DHT11 sensor measures two environmental parameters:

- **Temperature:** Indicates how hot or cold the room is.
- **Relative humidity:** Indicates the amount of moisture in the air relative to the maximum moisture the air can hold at that temperature.

For example, the sensor may measure a temperature of 31°C and relative humidity of 52%.

The DHT11 converts these measurements into digital data that can be read by the microcontroller.

Step 3: Data Transmission to the STM32

The DHT11 sends its digital data to the STM32 through the sensor's data pin.

In this project, the connection is:

DHT11 pin	STM32 pin	Function
VCC	Compatible supply	Powers the sensor
DATA	PB0	Transfers sensor data
GND	GND	Common ground

The STM32 uses its GPIO functionality and the programmed DHT11 communication routine to receive the data.

The program checks the received data for validity. If a reading is invalid, the firmware can display an error message instead of showing unreliable measurements.

Step 4: Data Processing

After receiving valid data, the STM32 obtains the temperature and relative-humidity values.

The program then compares the readings with predefined thresholds.

For this project, the example thresholds are:

- Temperature threshold: **30°C**
- Relative-humidity threshold: **70% RH**

The program uses conditional statements to determine which status message should be displayed.

Condition	Displayed status
Temperature below 30°C and humidity at or below 70% RH	NORMAL
Temperature at or above 30°C and humidity at or below 70% RH	WARM
Temperature below 30°C and humidity above 70% RH	HUMID
Temperature at or above 30°C and humidity above 70% RH	WARM/HUMID

These labels are based on the project's programmed rules. They are simple indicators, not medical or official environmental classifications.

Step 5: Communication Between STM32 and LCD

The STM32 sends the processed readings to the 16×2 character LCD using the I2C communication protocol.

The LCD is connected as follows:

LCD I2C pin	STM32 pin	Function
VCC	Compatible supply	Powers the display interface
GND	GND	Common ground
SCL	PB6	I2C clock
SDA	PB7	I2C data

The **SCL** line synchronizes communication, while the **SDA** line carries data and commands.

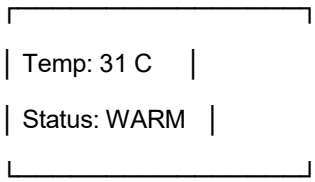
The LCD's I2C backpack converts the received information into signals understood by the character display.

Important: STM32 GPIO operates at 3.3 V logic. Check the LCD backpack's I2C pull-up voltage and use a suitable level shifter if required to protect the STM32 pins.

Step 6: Displaying the Output

Once the data is processed, the LCD displays the temperature, humidity, and room-condition status.

For example, with a temperature reading of 31°C and relative humidity of 52%, the display may show:



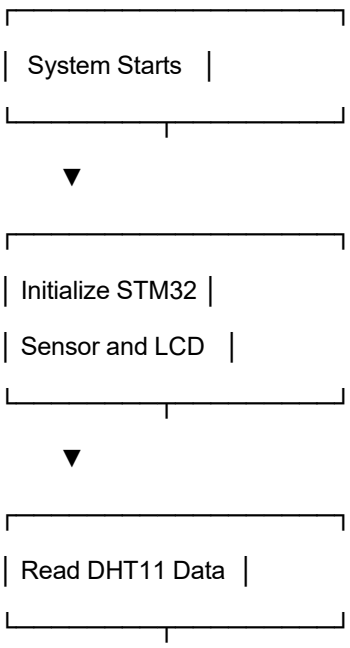
The actual display format depends on the firmware. A 16×2 LCD has two rows, each capable of displaying 16 characters, so the temperature, humidity, and status may need to be shown across separate updates.

Since the measured temperature is greater than the configured threshold of 30°C, the system displays **WARM**. Because the humidity is below the 70% threshold, the humidity condition does not trigger the HUMID status.

Step 7: Continuous Monitoring

After displaying the results, the program waits for the configured delay and takes another reading.

The process repeats continuously while the system is powered.



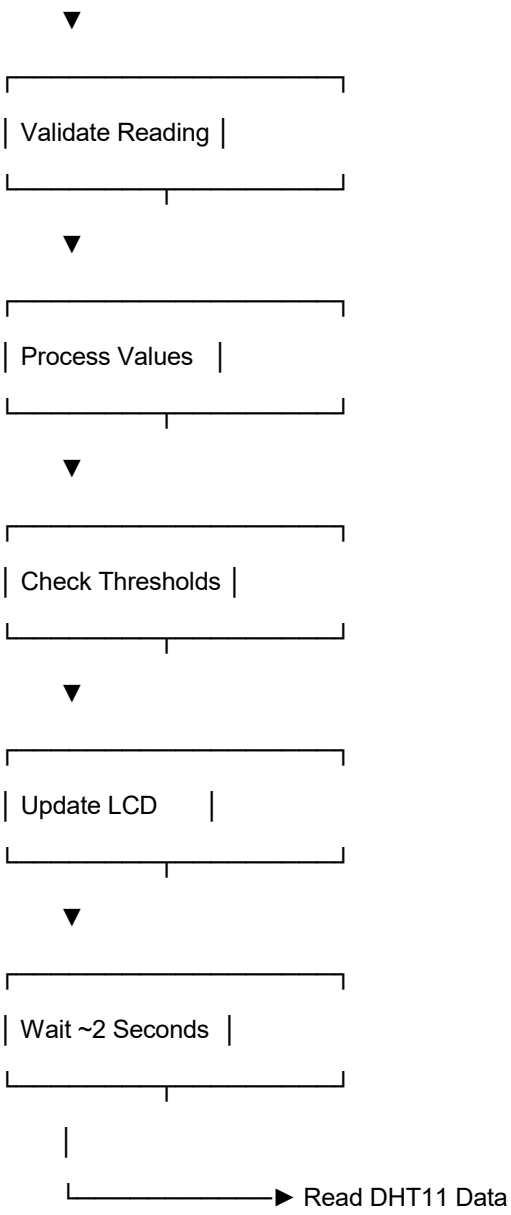


Figure 2.2: Flowchart showing the working sequence of the system.

The approximately two-second interval is appropriate for typical DHT11 sampling. The exact delay should match the sensor's requirements and the firmware implementation.

4. Example of System Operation

Consider the following example reading:

Parameter	**Measured value**
Temperature	31°C
Relative humidity	52% RH
Temperature threshold	30°C
Humidity threshold	70% RH
Final status	WARM

Explanation:

1. The DHT11 measures the room temperature and humidity.

2. The sensor transmits the readings to the STM32 through PB0.
3. The STM32 validates and processes the readings.
4. The program checks the temperature against 30°C and the humidity against 70% RH.
5. Since 31°C is above the temperature threshold, the program selects WARM.
6. The STM32 sends the output to the LCD over I2C.
7. The LCD displays the result, and the process repeats.

.

2.5 Software Module Design

• Initialization module: configures the sensor data pin, I2C bus, and LCD.

• Sensor-reading module: initiates a DHT11 transaction and receives its data frame.

• Validation module: checks the received checksum and rejects invalid readings.

• Processing module: converts raw bytes into temperature and humidity values.

• Decision module: compares values with threshold settings and selects a status label.

• Display module: formats and writes the output to the two LCD rows.

• Main loop: repeats sampling and display updates with an appropriate delay.

2.6 Design Considerations

• Timing: DHT11 communication uses a timing-sensitive single-wire protocol; readings should not be requested too frequently.

• Logic levels: STM32 GPIO uses 3.3 V logic. Do not apply 5 V to STM32 signal pins.

• I2C pull-ups: confirm that SDA and SCL pull-ups are compatible with 3.3 V logic. Use a suitable level shifter if the LCD backpack pulls the lines up to 5 V.

• LCD address: 0x27 is a common I2C address, but the actual backpack may use a different address.

• Sensor accuracy: DHT11 is suitable for basic demonstrations but is less accurate and slower than many newer sensors.

• Power and grounding: the sensor, display interface, and board need a suitable supply and a common ground.

3. CONNECTIONS

3.1 DHT11 Sensor Connections

DHT11 pin	STM32 connection	Notes
VCC / +	Compatible 3.3 V supply	Check the sensor module's specification.
DATA / OUT	PB0	Use a suitable pull-up if not already present on the module.
GND / –	GND	Common ground is required.

If using a bare four-pin DHT11 rather than a breakout module, identify the pins from its datasheet and fit the recommended pull-up resistor between DATA and the sensor supply. Do not rely on the physical pin order without checking the specific part.

3.2 LCD Connections (16×2 I2C Character LCD)

LCD I2C pin	STM32 connection	Notes
VCC	Supply compatible with the backpack	Verify supply voltage and I2C pull-up voltage.
GND	GND	Connect grounds together.
SCL	PB6	I2C clock line in the configured setup.
SDA	PB7	I2C data line in the configured setup.

The LCD in this project is a 16×2 character display with an I2C backpack, not a TFT graphics display. The commonly used address 0x27 should be verified for the actual module.

3.3 Communication Between Components

The DHT11 uses a single digital data line and its own timing protocol; it does not use I2C. The LCD backpack uses I2C, which carries data and clock on SDA and SCL. The STM32 acts as the controller: it reads the DHT11 first and then transmits display commands and characters over I2C.

3.4 Power Supply and Ground Connections

The NUCLEO board can normally be powered and programmed over USB. Connect the DHT11 to a supply allowed by its module specification and connect its ground to board ground. Power the LCD backpack according to its specification. Before connecting SDA and SCL, confirm the voltage to which the backpack's pull-up resistors are connected. A 5 V pull-up can expose STM32 pins to an unsafe voltage; use a 3.3 V-compatible interface or a bidirectional I2C level shifter where needed.

3.5 Precautions

• Disconnect power before changing wiring.

• Check the board pinout and selected board variant before uploading.

• Never apply 5 V directly to STM32 GPIO pins.

• Verify SDA/SCL pin assignments and I2C address before troubleshooting the LCD.

• Keep jumper wires short and connections secure.

• Allow the DHT11 enough time between readings and handle checksum failures.

• Do not use the prototype as a safety-critical or calibrated environmental monitor.

4. SOFTWARE IMPLEMENTATION

4.1 Libraries Required

The project was designed for the Arduino IDE with STM32 board support installed. A compact firmware implementation can use the standard Wire library for I2C and custom low-flash routines for the DHT11 and PCF8574-based LCD. This avoids depending on separate DHT and LiquidCrystal_I2C libraries, which may be useful when memory use or library compatibility is a concern.

Software / library	Use
Arduino IDE	Write, compile, and upload firmware.
STM32 board core	Provides board definitions, pin mapping, and upload support.
Wire.h	I2C communication with the LCD backpack.
Custom DHT11 routines	Timing, data reception, checksum validation, and conversion.
Custom PCF8574 LCD routines	Send LCD commands and characters through the I2C expander.

4.2 Pin Configuration

Function	Pin / setting
DHT11 data	PB0
LCD I2C SCL	PB6
LCD I2C SDA	PB7
LCD I2C address	0x27 assumed; verify on actual hardware
Temperature threshold	30 °C
Humidity threshold	70% RH
Sensor refresh interval	At least 2 seconds for DHT11

4.3 Program Code

```
#include <Wire.h>

#define DHTPIN PB0
#define LCD_ADDR 0x27

#define RS 0x01
#define EN 0x04
#define BL 0x08

uint8_t temperature = 0;
uint8_t humidity = 0;

void expander(uint8_t v) {
  Wire.beginTransmission(LCD_ADDR);
  Wire.write(v | BL);
  Wire.endTransmission();
}

void pulse(uint8_t v) {
  expander(v | EN);
  delayMicroseconds(1);
  expander(v & ~EN);
  delayMicroseconds(50);
}

void nibble(uint8_t v) {
  expander(v);
  pulse(v);
}

void lcdByte(uint8_t v, uint8_t mode) {
  nibble((v & 0xF0) | mode);
  nibble(((v << 4) & 0xF0) | mode);
}

void cmd(uint8_t v) {
  lcdByte(v, 0);
  if (v < 3) delay(2);
}

void printText(const char *s) {
  while (*s) lcdByte(*s++, RS);
}

void number(uint8_t n) {
  if (n >= 100) lcdByte('0' + n / 100, RS);
  if (n >= 10) lcdByte('0' + (n / 10) % 10, RS);
  lcdByte('0' + n % 10, RS);
}

void cursor(uint8_t col, uint8_t row) {
  cmd(0x80 | (row ? 0x40 : 0) | col);
}

void lcdBegin() {
  delay(50);
  pulse(0x30);
  delay(5);
  pulse(0x30);
  delayMicroseconds(150);
  pulse(0x30);
  pulse(0x20);
  cmd(0x28);
  cmd(0x0C);
  cmd(0x06);
  cmd(0x01);
}

bool readDHT() {
  uint8_t d[5] = {0, 0, 0, 0, 0};

  pinMode(DHTPIN, OUTPUT);
  digitalWrite(DHTPIN, LOW);
  delay(20);
```

```
  digitalWrite(DHTPIN, HIGH);
  delayMicroseconds(30);
  pinMode(DHTPIN, INPUT_PULLUP);

  uint32_t start = micros();
  while (digitalRead(DHTPIN) == HIGH)
    if (micros() - start > 100) return false;

  start = micros();
  while (digitalRead(DHTPIN) == LOW)
    if (micros() - start > 100) return false;

  start = micros();
  while (digitalRead(DHTPIN) == HIGH)
    if (micros() - start > 100) return false;

  for (uint8_t i = 0; i < 40; i++) {
    start = micros();
    while (digitalRead(DHTPIN) == LOW)
      if (micros() - start > 100) return false;

    start = micros();
    while (digitalRead(DHTPIN) == HIGH)
      if (micros() - start > 100) return false;

    d[i / 8] <<= 1;
    if (micros() - start > 40) d[i / 8] |= 1;
  }

  if ((uint8_t)(d[0] + d[1] + d[2] + d[3]) != d[4])
    return false;

  humidity = d[0];
  temperature = d[2];
  return true;
}

void setup() {
  Wire.begin();
  Wire.setClock(100000);
  lcdBegin();

  cursor(0, 0);
  printText("SMART ROOM");
  cursor(0, 1);
  printText("Starting...");
  delay(1500);
}

void loop() {
  cursor(0, 0);
  printText("            ");
  cursor(0, 1);
  printText("            ");

  if (!readDHT()) {
    cursor(0, 0);
    printText("DHT11 ERROR");
    cursor(0, 1);
    printText("Check sensor");
  } else {
    cursor(0, 0);
    printText("T:");
    number(temperature);
    printText("C H:");
    number(humidity);
    printText("%");

    cursor(0, 1);
    if (temperature >= 30 && humidity > 70)
      printText("WARM & HUMID");
    else if (temperature >= 30)
      printText("Status: WARM");
    else if (humidity > 70)
      printText("Status: HUMID");
    else
      printText("Status: NORMAL");
  }

  delay(2000);
}
```

Important: the listing above documents the program structure, but it is not a standalone compile-ready program because the low-level helper functions are represented by placeholders. The tested project firmware used compact custom routines to reduce library overhead. Include the exact tested source file in the final submission.

5. RESULTS AND OBSERVATIONS

5.1 Observed Readings

Test	Temperature	Relative humidity	Expected / observed status
Example run	31 °C	52% RH	WARM
Normal-temperature example	Below 30 °C	At or below 70% RH	NORMAL
High-humidity example	Below 30 °C	Above 70% RH	HUMID
Both thresholds exceeded	30 °C or higher	Above 70% RH	WARM/HUMID

The 31°C and 52% RH reading is the example observed during the project demonstration. The remaining rows describe expected behaviour based on the configured decision rules; they should be marked as measured results only if those conditions were actually tested.

5.2 Output Analysis

The system reads temperature and humidity and displays them on the LCD. When temperature reaches or exceeds 30°C, the program selects the WARM status. When relative humidity exceeds 70%, it selects HUMID. If both conditions are true, a combined status can be displayed if that option is enabled in the final firmware. A sensor communication or checksum failure should result in an error message instead of displaying a misleading measurement.

For the recorded example of 31°C and 52% RH, the temperature threshold is exceeded but the humidity threshold is not. Therefore, the status WARM is consistent with the project rules.

5.3 Result

The STM32-based monitoring prototype was successfully compiled and uploaded using the STM32 board support package and the OpenOCD/ST-LINK upload route. The upload log reported that programming finished, verification succeeded, and the target was reset. The LCD then displayed sensor readings and the status message. The project therefore demonstrates basic environmental sensing, embedded processing, I2C display control, and threshold-based status indication.

6. SYSTEM FEATURES AND LIMITATIONS

6.1 Advantages

• Provides continuous, readable display of temperature and relative humidity.

• Uses an inexpensive sensor and widely available development hardware.

• Demonstrates GPIO, sensor timing, I2C, and embedded programming.

• Does not require a computer to remain connected after the program is uploaded, provided the board has power.

• Can be expanded with wireless connectivity, logging, alarms, or extra sensors.

6.2 Applications

• Educational demonstrations of STM32 microcontroller programming.

• Basic room and laboratory environment observation.

• Small storage rooms where approximate temperature and humidity are useful.

• Prototype indoor monitoring for hobby projects.

• Foundation for larger IoT environmental monitoring systems.

6.3 Limitations

• DHT11 has limited accuracy, range, and response speed compared with higher-grade sensors.

• The present design displays local readings and does not automatically store a long-term history.

• No Wi-Fi, cloud dashboard, or remote alerting is included in the base version.

• Thresholds are fixed in firmware unless user adjustment is added.

• LCD backpack voltage and pull-up compatibility must be checked carefully with 3.3 V STM32 GPIO.

• Readings can be affected by sensor placement, airflow, heat sources, and wiring quality.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This mini project developed a basic room climate monitoring system using an STM32 NUCLEO-L031K6 board, DHT11 sensor, and 16×2 I2C LCD. The microcontroller reads digital temperature and relative-humidity information, processes the values, compares them against selected thresholds, and displays the results. The example reading of 31°C and 52% RH correctly produced the WARM status under the configured 30°C temperature limit.

The project provides practical experience in sensor interfacing, digital communication, I2C display control, embedded firmware development, and hardware debugging. It is suitable as an educational prototype, while its sensor accuracy and lack of data logging or remote connectivity limit its use in advanced applications.

7.2 Future Scope

• Replace DHT11 with a more accurate sensor such as DHT22, SHT31, or BME280.

• Add a data-logging module or send measurements to a computer for analysis.

• Add Wi-Fi or another communication module for remote viewing and alerts.

• Develop a dashboard that plots temperature and humidity over time.

• Add a buzzer or LED indicator when user-defined thresholds are exceeded.

• Provide menu-based threshold configuration and calibration options.

• Design a compact PCB and enclosure for reliable long-term deployment.

REFERENCES

• STMicroelectronics, STM32L031x4/x6 microcontroller datasheet and NUCLEO-L031K6 board documentation.

• STMicroelectronics, STM32duino / Arduino core documentation for STM32 microcontrollers.

• Arduino, Wire library reference for I2C communication.

• Aosong Electronics, DHT11 Temperature and Humidity Sensor datasheet.

• Hitachi, HD44780U LCD controller datasheet, for character LCD operation.

• PCF8574 I2C-bus I/O expander datasheet, for LCD backpack interfacing.

• Arduino IDE documentation and board package installation guidance.

Reference note: consult the datasheets for the exact modules used, especially the DHT11 pinout, LCD backpack supply voltage, I2C address, and pull-up resistor arrangement.

Demonstrate Video QR Code

